

ANKARA UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT
COM3035 OPERATING SYSTEMS
FALL 2025-26
LAB2

PURPOSE:

Design a multi-threaded program that processes seasonal performance data for teams in a football league.

The program should calculate the seasonal performance score for each team (e.g., based on the number of wins and draws), determine whether the team achieved a set league goal (e.g., qualifying for the Champions League or avoiding relegation), and write the results to an output file. In the output file, teams should be sorted by points. Finally, the output file should include the highest and lowest points in the league.

You will create a separate **thread** for each team in the league. Each thread will retrieve the data from a single team assigned to it, calculate the Team Score, and then pass it to the **semaphore** to update the highest and lowest points. In other words, the semaphore will be used to access the shared resource for team threads during the process of determining the highest and lowest points. Semaphore use is not required for team ranking.

Because threads run in parallel, multiple threads can attempt to modify shared data simultaneously. This can cause what's called a race condition, which can cause you to get incorrect results. (For example, if two threads try to update the high score at the same time, changes made by one thread might be lost.) Semaphores (or mutexes) are used to synchronize access to these shared resources.

REQUIREMENTS:

1. Input File Format (input.txt)

- The first line indicates the number of teams in the league.
- The second line indicates the number of teams qualifying for the Champions League.
- The third line indicates the number of teams relegated.

- Each subsequent line contains a Team Name followed by 5 statistical data items (Number of Wins, Number of Draws, Number of Losses, Goals Scored, and Goals Conceded, respectively).

Example:

```
12
1
1
Aberdeen 11 10 17 42 65
Celtic 30 5 3 95 30
Dundee 8 13 17 35 62
Hearts 19 8 11 60 45
Hibernian 12 11 15 48 58
Kilmarnock 16 9 13 52 50
Livingston 5 8 25 28 85
Motherwell 10 12 16 40 60
Rangers 28 6 4 88 32
RossCounty 8 9 21 38 70
St.Johnstone 7 10 21 30 68
St.Mirren 14 10 14 45 55
```

2. Output File Format (output.txt)

- For each team, listed in order of points: Team Name, Team Points, and Status (e.g., Qualified/Relegated).
 - Qualified: Team(s) that qualified for the Champions League
 - Relegated: Team(s) that were relegated
 - Teams that did not qualify for the Champions League but were not relegated will not be assigned any status.

Example:

```
Celtic 95 - Qualified
Rangers 90
Hearts 65
Kilmarnock 57
St.Mirren 52
Hibernian 47
Aberdeen 43
Motherwell 42
Dundee 37
RossCounty 33
St.Johnstone 31
Livingston 23 - Relegated

Highest point: 95
Lowest point: 23
```

RULES:

- The teams that will advance to the Champions League and the teams that will be relegated will be determined based on their points ranking.
 - Teams will be ranked from highest to lowest based on their points, and the number of teams advancing to the Champions League, as specified in the input file, will be selected for this league, starting with the highest points.
 - The number of teams relegated, as specified in the input file, will be determined in order, starting with the lowest points.
- Teams' points will be calculated based on their number of wins and draws. Each win will be considered 3 points, and each draw will be considered 1 point. Losses will not affect the points calculation.
 - If teams are tied on points, their goal difference (Goals Scored – Goals Conceded) will be calculated, and the ranking will be determined accordingly. The goal difference is a rule based on the difference in goals scored and conceded between teams that are tied on points to determine their superiority over each other.
 - If the points and goal difference of the teams are equal, the Goals Scored information will be taken into account and the team with the highest Goals Scored value will be placed first in the rankings.

SUBMISSION:

- Your code files must be in C programming and must run from the terminal on Linux-based operating systems.
- Your code file must be named `student_number.c` (e.g., `18888888.c`). Points will be deducted for incorrectly named submissions.
- You must only upload the `student_number.c` file to the system. You do not need to upload it as a Zip file.
- When running your program, you must read input data from a file such as `input1.txt` and write output to a file such as `output1.txt`. Using `fopen`, `fread`, and `fwrite` within your program file is strictly prohibited! Reading from and writing to the file will be done entirely with terminal commands.
- Your program output must match the format in the shared sample input-output files exactly!
- Your code file must be compiled and run from the terminal with the following commands:
 - **`gcc studentnumber.c -o studentnumber -lpthread`**

- This command compiles your C code file and creates an output file named `yourprogramname`. For multi-threading support, use the `-lpthread` flag.
- ***./studentnumber <input1.txt>out1.txt***
 - This command allows you to run your compiled code file, taking the corresponding `input1.txt` file as input, and print the results to the file named `out1.txt`.
- ***diff -w out1.txt output1.txt***
 - Use this command to compare your output (`out1.txt`) with the intended output file (`output1.txt`). If no warning appears on the screen after entering this command, your program is working correctly for these values. If you see a warning in the command prompt after entering the command, this indicates a problem with your output, and you should correct your code.
- Your program code will be checked for copying. Code detected as unoriginal (taken from the internet or copied, etc.) will be evaluated as zero.

TÜRKÇE:

AMAÇ:

Futbol ligindeki takımların sezonluk performans verilerini işleyen çoklu iş parçacıklı (multi-thread) bir program tasarlayınız.

Program, her takım için sezonluk başarı puanını (örneğin, galibiyet ve beraberlik sayısına göre bir formülle) hesaplamalı, takımın ligde belirlenen bir hedefi (örneğin, Şampiyonlar Ligi'ne katılma veya küme düşmeme) başarıp başaramadığını belirlemeli ve sonuçları bir çıktı dosyasına yazmalıdır. Çıktı dosyasında takımlar puana göre sıralanmış olmalıdır. Çıktı dosyasına son olarak ligdeki en yüksek puan ile en düşük puan bilgileri de eklenmelidir.

Ligdeki her takım için ayrı bir **iş parçacığı (thread)** oluşturacaksınız. Her iş parçacığı, kendisine atanan tek bir takımın verisini alıp, Takım Puanını hesaplayacak ve en yüksek, en düşük puanları güncellemek için **semaphore**'a geçecektir. Yani en yüksek ve en düşük puan belirlenmesi sürecinde takım thread'lerinin ortak erişim alanına (shared resource) girişlerinde semaphore kullanılacaktır. Takım sıralaması için semaphore kullanımı gerekli değildir.

İş parçacıkları paralel çalıştığı için, birden fazla iş parçacığı aynı anda paylaşılan bir veriyi değiştirmeye çalışabilir. Bu, yarış durumu (Race Condition) denen hataya yol açar ve yanlış sonuçlar almanıza neden olur (Örneğin, iki iş parçacığı aynı anda en yüksek puanı güncellemeye çalışırsa, birinin yaptığı değişiklik kaybolabilir). **Semaphore'lar (ya da Mutex)**, bu paylaşılan kaynaklara erişimi senkronize etmek için kullanılır.

GEREKSİNİMLER:

1. Girdi Dosyası Formatı (input.txt)

- İlk satır, ligdeki takım sayısını belirtir.
- İkinci satır, Şampiyonlar Ligi'ne gidecek takım sayısını belirtir.
- Üçüncü satır, küme düşecek takım sayısını belirtir.
- Takip eden her satır, bir Takım Adı ve ardından 5 adet istatistik verisini (sırasıyla Galibiyet Sayısı, Beraberlik Sayısı, Mağlubiyet Sayısı, Atılan Gol, Yenilen Gol) içerir.

Örnek:

```
12
1
1
Aberdeen 11 10 17 42 65
Celtic 30 5 3 95 30
Dundee 8 13 17 35 62
Hearts 19 8 11 60 45
Hibernian 12 11 15 48 58
Kilmarnock 16 9 13 52 50
Livingston 5 8 25 28 85
Motherwell 10 12 16 40 60
Rangers 28 6 4 88 32
RossCounty 8 9 21 38 70
St.Johnstone 7 10 21 30 68
St.Mirren 14 10 14 45 55
```

2. Çıktı Dosyası Formatı (output.txt)

- Puan sırasına göre sıralanmış olarak her takım için: Takım Adı, Takım Puanı ve Durumu (Örn: Qualified/Relegated).
 - Qualified: Şampiyonlar Ligi'ne gitmeye hak kazanan takım(lar)
 - Relegated: Küme düşen takım(lar)
 - Şampiyonlar Ligi'ne gidemeyen ama küme de düşmeyen takımlara herhangi bir durum belirtilmeyecektir.

Örnek:

```
Celtic 95 - Qualified
Rangers 90
Hearts 65
Kilmarnock 57
St.Mirren 52
Hibernian 47
Aberdeen 43
Motherwell 42
Dundee 37
RossCounty 33
St.Johnstone 31
Livingston 23 - Relegated

Highest point: 95
Lowest point: 23
```

KURALLAR:

- Ligden Şampiyonlar Ligi'ne gidecek ve küme düşecek takım(lar) puan sıralamasına göre belirlenecektir.

- Takımlar puanlarına göre büyükten küçüğe sıralanacak ve girdi dosyasında verilen Şampiyonlar Ligi'ne gidecek takım sayısı kadarı, en yüksek puanlıdan başlayarak sırasıyla bu lig için seçilecektir.
- Girdi dosyasında verilen küme düşecek takım sayısı kadarı da en düşük puanlı takımdan başlayarak sırasıyla belirlenecektir.
- Takımların puanları aldıkları galibiyet ve beraberlik sayıları üzerinden hesaplanacaktır. Her galibiyet 3 puan, her beraberlik 1 puan kabul edilecektir. Mağlubiyetlerin puan hesaplamasına bir etkisi olmayacaktır.
 - Takımların puanlarının eşit olması durumunda takımların averaj durumu (Atılan Gol – Yenilen Gol) hesaplanacak ve buna göre sıralama belirlenecektir. Averaj, aralarında puan eşitliği bulunan takımların birbirlerine göre üstünlüğünü belirlemek için attıkları ve yedikleri gollerin farkına dayalı kuraldır.
 - Takımların puanları ve averajlarının eşit olması durumunda Atılan Gol bilgisine bakılacak ve en yüksek Atılan Gol değerine sahip takım sıralamada önde yer alacaktır.

TESLİMAT:

- Kod dosyalarınız C programlama dilinde olmalı ve Linux tabanlı işletim sistemlerinde terminal üzerinden çalışmalıdır.
- Kod dosyanız öğrenci_numarası.c (örn: 18888888.c) olarak adlandırılmalıdır. Doğru isimlendirilmeyen gönderimlerden puan kırılabilecektir.
- Sisteme sadece öğrenci_numarası.c dosyasını yüklemelisiniz. Zip dosyası olarak yüklemenize gerek yoktur.
- Programınızı çalıştırırken girdi verilerini input1.txt gibi bir dosyadan okumalısınız ve çıktıları da output1.txt gibi bir dosyaya yazmalısınız. **Program dosyanız içinde fopen, fread, fwrite kullanımı kesinlikle yasaktır!** Dosyadan okuma ve dosyaya yazma işlemleri tamamen terminal komutları ile yapılacaktır.
- **Program çıktılarınız paylaşılan örnek input-output dosyalarındaki formata bire bir uymalıdır!**
- Kod dosyanız terminal üzerinde aşağıdaki komutlarla derlenip çalıştırılmalıdır:
 - ***gcc studentnumber.c -o studentnumber -lpthread***
 - Bu komut c kod dosyanızı derler ve yourprogramname adıyla bir output dosyası oluşturur. Çoklu iş parçacığı desteği için -lpthread bayrağını kullanınız.
 - ***./studentnumber <input1.txt>out1.txt***

- Bu komut, derlenen kod dosyanızı ilgili input1.txt dosyasını girdi olarak alıp çalıştırmanızı ve sonuçları da out1.txt olarak adlandırılan dosyaya yazdırmanızı sağlar.
- *diff -w out1.txt output1.txt*
 - Bu komutu kullanarak kendi çıktınız (out1.txt) ile olması gereken çıktı dosyasını (output1.txt) karşılaştırınız. Bu komutu girdikten sonra ekranda bir uyarı çıkmıyorsa, programınız bu değerler için doğru çalışıyor demektir. Eğer komutu girdikten sonra komut sisteminde uyarı görüyorsanız bu çıktınızda problem olduğunu gösterir, kodunuzu düzeltmeniz gerekmektedir.
- **Program kodlarınız kopya kontrolünden geçirilecektir.** Özgün olmadığı (internette alınan ya da kopya vs.) tespit edilen kodlar sıfır olarak değerlendirilecektir.